

Student Manual

Implementing a Tiny Dreamer-v2 Style RSSM

Categorical latent world model for hidden-velocity CartPole

Read this first. Your task is to implement the world-model part of Dreamer: a recurrent state-space model (RSSM). This document deliberately does **not** give you the RSSM code. It gives you the architecture, tensor shapes, formulas, method contracts, and debugging checks you need to write it yourself.

Assumed background. You have already implemented and trained A2C. That helps because you know PyTorch modules, optimizers, rollouts, losses, and train/eval loops. But the RSSM is not an actor-critic. It does not choose actions and it does not estimate returns.

What you will build. You will build a supervised sequence model. It receives short chunks of CartPole data, where velocities are hidden, and learns to predict the next visible observation, reward, and whether the episode continues.

0. The whole project in one paragraph

CartPole normally gives four numbers: cart position, cart velocity, pole angle, and pole angular velocity. In this notebook the model only sees cart position and pole angle. Therefore, the current observation alone is not enough to predict the next observation well. The model needs memory. The RSSM provides this memory through a deterministic recurrent state h and a stochastic categorical latent state z . During training, the posterior is allowed to look at the real next observation. The prior is not. The goal is to train the prior so that it can later imagine forward without seeing future observations.

1. What is different from A2C?

In A2C, you used data to improve an agent. Here, you use data to learn a model of the environment. This distinction matters. Do not add an actor, critic, advantage, entropy bonus, or policy-gradient loss to this notebook.

A2C idea	RSSM version in this notebook	What to remember
Actor $\pi(a s)$	No actor	Actions are already in the dataset. The RSSM receives actions as inputs.
Critic $V(s)$	No critic	The RSSM predicts immediate reward, not discounted return.
Rollout buffer	SequenceDataset	Training samples are short supervised sequence chunks.
Hidden state in RNN policy	Deterministic RSSM state h	h stores history, especially information about hidden velocities.
Policy/value losses	Prediction losses plus KL	The objective is model learning, not policy improvement.

Critical warning. If you treat this like A2C, you will implement the wrong algorithm. This notebook is a world-model pre-training exercise.

2. Data: what the model sees and predicts

The environment gives the full CartPole state, but the model only receives two entries: cart position and pole angle. This helper code is allowed because it is not the RSSM itself.

```
VISIBLE_STATE_INDICES = np.array([0, 2])

def visible_state(full_state):
    return full_state[VISIBLE_STATE_INDICES].astype(np.float32)

def one_hot(action, action_size=2):
    action_vector = np.zeros(action_size, dtype=np.float32)
    action_vector[action] = 1.0
    return action_vector
```

Each training item is a chunk of length T . There are T actions but $T+1$ observations, because every action predicts the next observation.

Name	Shape	Meaning
observations	$[B, T + 1, 2]$	Visible observations $o_0, \dots, o_T$.
actions	$[B, T, 2]$	One-hot actions $a_0, \dots, a_{T-1}$.
rewards	$[B, T, 1]$	Reward after each action. For CartPole this is usually 1 until termination.

Name	Shape	Meaning
continues	[B, T, 1]	1 if the episode continues after the step; 0 if it ended.

Alignment rule. At time step t , use action `actions[:, t]` and predict observations `[:, t + 1]`, rewards `[:, t]`, and continues `[:, t]`. If your model learns nothing, check this first.

3. The two states inside the RSSM

The RSSM state has two parts. Keep these separate in your head and in your code.

State	Typical name	Shape	Role
Deterministic recurrent state	h	[B, deterministic_size]	A GRU memory that summarizes past observations and actions.
Stochastic categorical state	z	[B, categorical_size, class_size]	A sampled one-hot latent representation at the current step.
Flattened stochastic state	flat_z	[B, categorical_size * class_size]	The version of z used in Linear layers and concatenations.

Hyperparameter	Example value	Meaning
categorical_size	4	Number of separate categorical variables.
class_size	8	Number of classes inside each categorical variable.
stochastic_size	32	categorical_size * class_size after flattening.
deterministic_size	8	Size of GRU hidden state h .
hidden_size	16	Width of small MLPs used inside the model.

This means the latent state is not one class out of 32. It is 4 separate categorical variables, each choosing one class out of 8. Softmax and KL must therefore operate over the last dimension only, the class dimension.

4. Prior and posterior: the central distinction

The RSSM has two distributions over the categorical latent state z . They have the same output shape but different inputs.

Distribution	Input	Output	Used when?	Interpretation
Prior $p(z_t h_t)$	h_t only	logits [B, C, K]	Imagination and prior prediction	What the model expects before seeing the next observation.
Posterior $q(z_t h_t, o_t)$	h_t plus real next observation	logits [B, C, K]	Training and warmup	What the model infers after seeing what actually happened.

The posterior is easier because it sees the answer: the next observation. The prior is more important because it is the version that can run without future observations. The KL loss teaches the prior to match the posterior.

Do not leak information. Only the posterior receives observations `[:, t + 1]`. If the prior receives the future observation, your rollout test becomes meaningless.

5. Required neural networks

Your model should contain the following modules. You must write the code yourself, but these contracts are fixed.

Component	Input shape	Output shape	Purpose
action_stack	[B, stochastic_size + action_size]	[B, hidden_size]	Preprocess previous latent and current action before the GRU.
GRUCell	input [B, hidden_size], old h [B, H]	new h [B, H]	Update deterministic memory.
prior_model	[B, H]	[B, C*K]	Predict categorical logits from h only. Reshape to [B,C,K].
posterior_model	[B, H + observation_size]	[B, C*K]	Predict categorical logits from h and next observation. Reshape to [B,C,K].
prediction_model	[B, H + C*K]	[B, hidden_size]	Build features for prediction heads.
observation_head	[B, hidden_size]	[B, 2]	Predict next visible CartPole observation.

Component	Input shape	Output shape	Purpose
reward_head	[B, hidden_size]	[B, 1]	Predict immediate reward.
continue_head	[B, hidden_size]	[B, 1]	Predict raw logit for whether the episode continues.

Use small networks: one Linear layer, ReLU, and another Linear layer is enough for the prior and posterior. This notebook is not about achieving a benchmark score. It is about understanding the mechanics.

6. Required methods and their contracts

Your class should expose these methods. The table tells you what each method must receive and return. It intentionally does not give the implementation.

Method	Input	Return	Key requirements
initial	batch_size, device	flat_z [B,C*K], h [B,H]	Both should start as zeros.
logits_to_shape	logits [B,C*K]	[B,C,K]	Do this before softmax, sampling, mode, or KL.
flatten_latent	z [B,C,K]	flat_z [B,C*K]	Do this before Linear layers and concatenation with actions.
sample_straight_through	logits [B,C,K]	z [B,C,K]	Forward pass should be one-hot; gradient should flow through probabilities.
mode_one_hot	logits [B,C,K]	z [B,C,K]	Use argmax over K for deterministic evaluation/rollout.
prediction_heads	flat_z [B,C*K], h [B,H]	dict with observation, reward, continue_logit	Shapes must be [B,2], [B,1], [B,1].
observe / forward	observations [B,T+1,2], actions [B,T,2]	stacked logits and predictions	Loop over T steps. Use posterior state for the next recurrent step.

7. The recurrent update algorithm

At each time step t , implement the following sequence exactly. This is the core of the RSSM.

- 1 Start the step with the previous posterior latent `flat_z` and previous deterministic state `h`. At $t = 0$ both are zero.
- 2 Concatenate `flat_z` with the current one-hot action `actions[:, t]`.
- 3 Pass this concatenated vector through `action_stack`.
- 4 Update `h` with the GRUCell.
- 5 Compute prior logits from `h` only, then reshape them to `[B,C,K]`.
- 6 Sample a prior latent from the prior logits and flatten it. Use it to make prior predictions.
- 7 Compute posterior logits from the concatenation of `h` and the real next observation `observations[:, t + 1]`.
- 8 Sample the posterior latent and flatten it. Store it as the latent state for the next time step.
- 9 Use the posterior latent to make posterior predictions.
- 10 Store prior logits, posterior logits, prior predictions, and posterior predictions for this time step.

The subtle point is step 8: the next recurrent step should use the posterior latent during training. That lets the model stay grounded in the real trajectory while it learns.

8. Categorical sampling: what you must implement

For each categorical variable, the logits define a distribution over K classes. You need two versions of the latent sample.

Version	When to use it	What it should do
Straight-through sample	Training	Sample a one-hot vector in the forward pass, but use the softmax probabilities for the backward gradient.
Mode one-hot	Evaluation and plotting	Choose argmax over the class dimension K and convert it to one-hot. No randomness.

Shape rule: sampling is independent for each of the C categorical variables. Therefore, the result has shape `[B,C,K]`, and every `[K]` slice should contain exactly one active class in the forward pass.

Conceptual straight-through formula: $z = \text{one_hot_sample} + \text{probs} - \text{stop_gradient}(\text{probs})$. In the forward pass this equals the one-hot sample. In the backward pass gradients flow as if probabilities had been used.

9. Prediction heads

Predictions are made from the concatenation of deterministic memory h and a flattened latent $\text{flat_}z$. Use the same prediction heads for prior and posterior latents. This parameter sharing is important: the only difference should be which latent distribution produced z .

Prediction	Input features	Output	Loss target
next visible observation	$[\text{flat_}z, h]$	$[B, 2]$	$\text{observations[:, t+1]}$
reward	$[\text{flat_}z, h]$	$[B, 1]$	rewards[:, t]
continuation logit	$[\text{flat_}z, h]$	$[B, 1]$	continues[:, t]

For this didactic notebook, predict observation and reward directly as real-valued numbers. Do not implement Gaussian distributions unless explicitly asked later. Keep the focus on the categorical latent RSSM.

10. What forward/observe must return

After looping over the sequence, stack all time-step outputs along the time dimension. Your output dictionary should have at least these entries:

Key	Shape	Meaning
prior_logits	$[B, T, C, K]$	Prior categorical logits at each time step.
posterior_logits	$[B, T, C, K]$	Posterior categorical logits at each time step.
prior_predictions["observation"]	$[B, T, 2]$	Predictions made from prior samples.
prior_predictions["reward"]	$[B, T, 1]$	Reward predictions made from prior samples.
prior_predictions["continue_logit"]	$[B, T, 1]$	Continuation logits made from prior samples.
posterior_predictions[...]	same shapes	The same prediction heads applied to posterior samples.

Helper code for stacking a list of prediction dictionaries is acceptable:

```
def stack_predictions(predictions):
    return {
        key: torch.stack([prediction[key] for prediction in predictions], dim=1)
        for key in predictions[0]}

```

11. Losses: exact specification

The RSSM objective combines prediction losses and a KL loss. You should implement these yourself from the specification below.

Loss term	Formula / requirement	Target
Observation loss	Mean squared error	$\text{prediction["observation"]} \text{ vs } \text{observations[:, 1:]}$
Reward loss	Mean squared error	$\text{prediction["reward"]} \text{ vs } \text{rewards}$
Continuation loss	Binary cross entropy with logits	$\text{prediction["continue_logit"]} \text{ vs } \text{continues}$
Prediction loss	$\text{observation_loss} + \text{reward_loss} + \text{continuation_loss}$	Compute once for posterior predictions and once for prior predictions.

Do not use cross entropy for the latent categories. There is no true label for z . The categories are learned hidden variables.

Categorical KL

The KL term trains the prior distribution to match the posterior distribution. Both inputs are logits with shape $[B, T, C, K]$.

For each batch item, time step, and categorical variable, compute:

$$\text{KL}(q \parallel p) = \sum_k q_k * (\log q_k - \log p_k)$$

Then sum over K , sum over C , and average over B and T . In implementation terms: softmax/log-softmax must operate over the final class dimension K , not over the flattened $C*K$ dimension.

The total loss should be:

`total = posterior_loss + prior_scale * prior_loss + kl_scale * kl_loss`

Recommended scale	Value	Reason
prior_scale	0.5	Makes the prior useful without overwhelming the easier posterior learning signal.
kl_scale	0.1	Keeps posterior and prior close while avoiding early collapse.

12. Training and testing

The training loop should look familiar from A2C, but the objective is supervised world-model learning.

- 1 Move batch tensors to the device.
- 2 Call the model on observations and actions.
- 3 Compute the RSSM loss described above.
- 4 Backpropagate.
- 5 Step the optimizer.
- 6 Zero gradients.

Track at least these metrics: total loss, KL, posterior observation MSE, prior observation MSE, reward MSE, and continuation accuracy.

Metric	How to interpret it
posterior observation MSE	Should usually be lower than prior MSE because the posterior sees the next observation.
prior observation MSE	The most important prediction metric for imagination.
KL	Must be finite and non-negative. If it is negative, the formula is wrong.
continue accuracy	Can be high because most CartPole steps continue. Do not overinterpret it.

13. Prior rollout: testing imagination

After training, test whether the model can roll forward without seeing future observations. First warm it up with a few real observations using the posterior. Then switch to the prior and use only recorded future actions.

Phase	What the model receives	Which latent distribution to use
Warmup	real observations and recorded actions	posterior $q(z h, o)$
Rollout	recorded actions only	prior $p(z h)$

Warmup algorithm: initialize `flat_z` and `h` to zero, then for each warmup action update `h`, infer the posterior using the real next observation, choose the posterior mode one-hot, and set `flat_z` to that posterior latent.

Rollout algorithm: for each future action update `h`, compute the prior from `h` only, choose the prior mode one-hot, set `flat_z` to that prior latent, predict the visible observation, and store the prediction.

Do not expect perfect long-horizon predictions. Errors accumulate. The goal is that short-horizon predictions roughly track the visible CartPole trajectory better than a random or flat baseline.

14. Recommended implementation order

Implement in this order. Do not write the whole model at once and then debug everything together.

Step	Implement	Check before moving on
1	Environment helpers: <code>reset</code> , <code>step</code> , <code>visible_state</code> , <code>one_hot</code>	A full state has 4 values; <code>visible_state</code> has 2.
2	<code>collect_episodes</code>	Each episode stores observations, actions, rewards, continues, and <code>full_states</code> .
3	<code>SequenceDataset</code>	One batch has observations <code>[B,T+1,2]</code> , actions <code>[B,T,2]</code> , rewards <code>[B,T,1]</code> .
4	<code>TinyRSSM.__init__</code>	Print the model. Prior/posterior final layers output $C \times K$ values.
5	<code>logits_to_shape</code> , <code>flatten_latent</code> , <code>initial</code>	Shapes are <code>[B,C,K]</code> , <code>[B,C*K]</code> , and <code>[B,C*K]/[B,H]</code> .
6	<code>sample_straight_through</code> and <code>mode_one_hot</code>	Samples are one-hot along the K dimension.
7	<code>prediction_heads</code>	Outputs observation <code>[B,2]</code> , reward <code>[B,1]</code> , <code>continue_logit</code> <code>[B,1]</code> .

Step	Implement	Check before moving on
8	observe/forward	Returned tensors are stacked over time: [B,T,...].
9	prediction losses and categorical KL	Loss is finite; KL is non-negative.
10	train/test loop	Observation MSE should decrease over epochs.
11	prior_rollout and plots	Short rollouts should be better than random flat lines.

15. Debugging checklist

Use this checklist before asking whether the model architecture is wrong. Most failures are shape or alignment errors.

Problem	Likely cause	Fix
Loss is NaN	Too high learning rate, bad KL computation, invalid probabilities	Start with lr=1e-3, use log_softmax, check tensors for NaN.
KL is negative	KL formula or reduction is wrong	Use $q * (\log q - \log p)$, then sum over K and C.
Prior rollout is unrealistically good	Prior accidentally sees future observation	Check that only posterior_model receives observations[:, t+1].
Shapes fail at concat	Forgot to flatten z before Linear layers	Use flat_z before concatenating with action or h.
Softmax seems wrong	Applied softmax after flattening	Keep logits as [B,T,C,K] or [B,C,K] for softmax/KL.
Continue accuracy is odd	Used sigmoid before BCE-with-logits or wrong target shape	Pass raw logits to BCE; target shape must be [B,T,1].
Posterior works but prior fails	KL too weak, prior loss too weak, or rollout leak in training	Check KL scale, prior_scale, and that prior predictions are part of total loss.

16. What counts as a finished implementation?

Your implementation is finished when all of the following are true:

- The notebook runs from top to bottom without shape errors.
- The model prints with the expected components: action_stack, GRUCell, prior_model, posterior_model, prediction_model, and three heads.
- A batch has the expected shapes: observations [B,T+1,2], actions [B,T,2], rewards [B,T,1], continues [B,T,1].
- Prior and posterior logits have shape [B,T,C,K].
- Latent samples are one-hot along K before flattening.
- Total loss decreases over training, at least roughly.
- KL is finite and non-negative.
- Prior rollout uses no future observations after warmup.
- The final plot compares prior predictions against the real visible CartPole trajectory.

Final conceptual test. Explain in your own words why the posterior is allowed to see the next observation during training, but the prior is the model we need for imagination. If you cannot explain that, the code may run, but the RSSM is not yet understood.

17. What this leaves out

This is Dreamer-like because it learns a latent recurrent world model with prior and posterior latent states. It is not a full Dreamer agent. There is no actor, critic, imagination objective, image encoder, or image decoder. The point is to isolate the RSSM mechanics before adding policy learning.

A later Dreamer implementation would add an actor and critic trained on imagined trajectories. That is a separate step. Do not mix it into this notebook.